

Navigation Centrale

Documentation du Projet

- [Accueil](#) | [Documentation Centralisée](#)

Démarrage Rapide

- [Guide Installation](#) | [Guide Développement](#)

API & Intégration

- [Documentation API](#) | [API Technique](#)

Déploiement

- [Guide Déploiement](#) | [Déploiement Technique](#)

Outils Développement

- [Scripts et Makefiles](#) | [Documentation Technique](#)

Documentation Technique

- [Architecture](#) | [Base de Données](#) | [Sécurité](#) | [Performance](#)
-

Architecture Microservices JobbingTrack

Documentation complète de l'architecture technique de JobbingTrack.

Navigation Centrale

Documentation du Projet

- [Accueil](#) | [Documentation Centralisée](#)

Démarrage Rapide

- [Guide Installation](#) | [Guide Développement](#)

API & Intégration

- [Documentation API](#) | [API Technique](#)

Déploiement

- [Guide Déploiement](#) | [Déploiement Technique](#)

Outils Développement

- [Scripts et Makefiles](#) | [Documentation Technique](#)

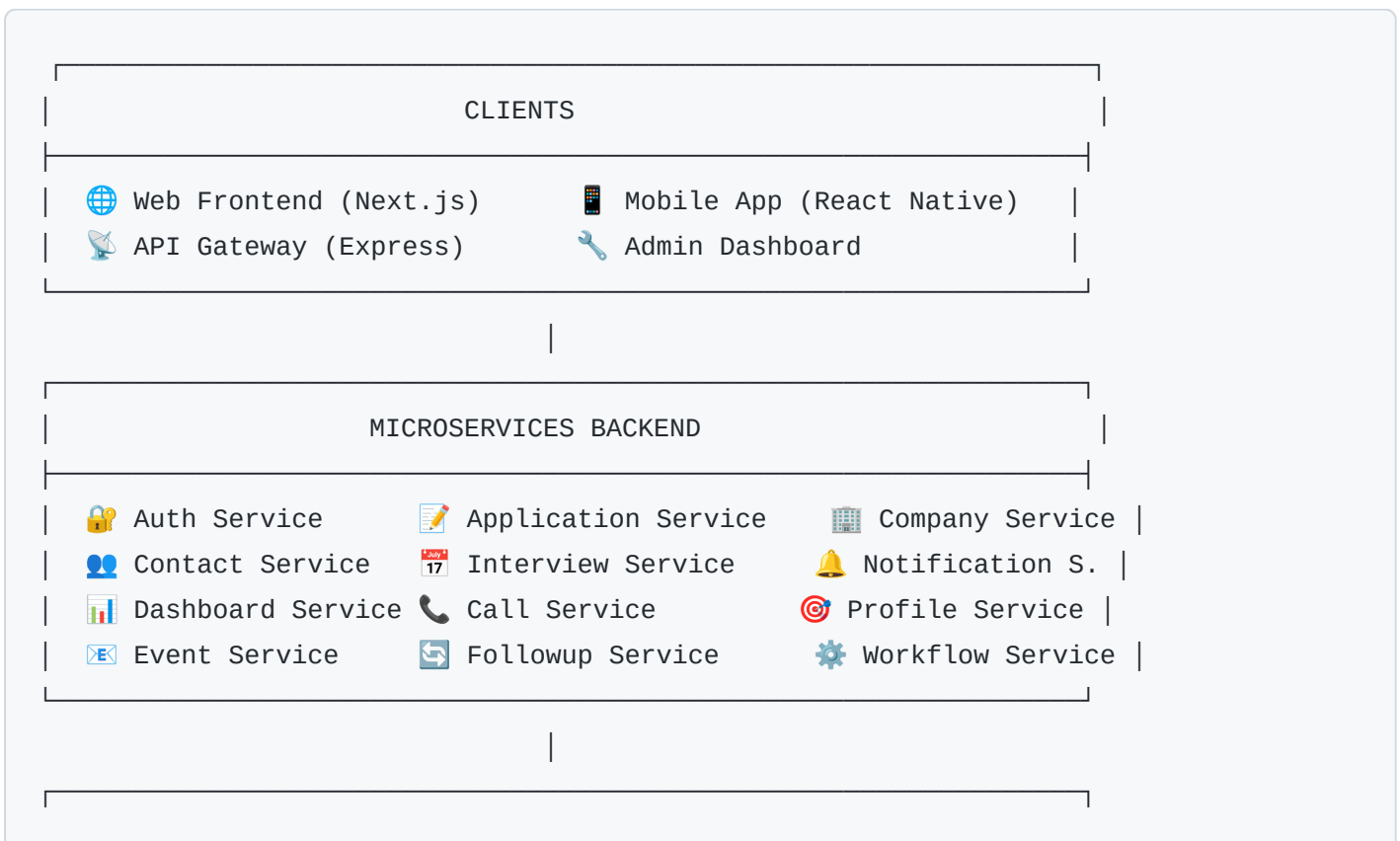
Documentation Technique

- [Base de Données](#) | [Sécurité](#) | [Performance](#)

Vue d'Ensemble

Vue d'Ensemble

JobbingTrack est construit sur une **architecture microservices moderne** avec séparation claire des responsabilités et scalabilité horizontale.



INFRASTRUCTURE



PostgreSQL 15



Redis 7



Prometheus



Grafana



Jaeger



Docker



Nginx Proxy



SSL/TLS



Logging

Principes d'Architecture

1. Séparation des Responsabilités

Chaque microservice a une responsabilité unique et bien définie :

Service	Responsabilité	Port	Base de Données
API Gateway	Routage, authentification, rate limiting	3000	-
Auth Service	JWT, utilisateurs, sessions	3001	PostgreSQL
Application Service	CRUD candidatures, timeline	3002	PostgreSQL
Company Service	Gestion entreprises, secteurs	3003	PostgreSQL
Contact Service	Carnet contacts professionnels	3004	PostgreSQL
Interview Service	Planning entretiens, feedback	3005	PostgreSQL
Notification Service	Emails, rappels automatiques	3006	Redis + SMTP
Dashboard Service	KPIs, analytics, statistiques	3007	PostgreSQL
Call Service	Gestion appels, notes	3008	PostgreSQL
Profile Service	Profils utilisateurs étendus	3009	PostgreSQL
Event Service	Événements système	3011	PostgreSQL
Followup Service	Relances automatiques	3012	PostgreSQL
Workflow Service	Workflows métier	3013	PostgreSQL

2. Communication Inter-Services

- **Protocole** : HTTP/REST avec JSON
- **Authentification** : JWT tokens partagés
- **Découverte** : Configuration statique via variables d'environnement
- **Résilience** : Timeout, retry, circuit breaker

3. Gestion des Données

Base de Données Principale

- **PostgreSQL 15** avec extensions avancées
- **Schéma multi-tenant** par utilisateur
- **Indexes optimisés** pour les requêtes fréquentes
- **Migrations automatiques** avec Prisma

Cache et Sessions

- **Redis 7** pour le cache et les sessions
- **TTL configurable** selon le type de données
- **Invalidation intelligente** lors des modifications



Stack Technique

Backend (Microservices)

```
{
  "runtime": "Node.js 20 LTS",
  "framework": "Express.js 4.x",
  "orm": "Prisma 6.x",
  "database": "PostgreSQL 15",
  "cache": "Redis 7",
  "auth": "JWT (jsonwebtoken)",
  "validation": "Zod",
  "logging": "Winston",
  "monitoring": "Prometheus + Grafana",
  "tracing": "Jaeger"
}
```

Frontend (Next.js)

```
{
  "framework": "Next.js 14",
  "language": "TypeScript 5.x",
  "styling": "Tailwind CSS 3.x",
  "ui": "Radix UI + shadcn/ui",
  "state": "Zustand",
  "queries": "React Query",
  "forms": "React Hook Form",
  "charts": "Recharts",
  "testing": "Playwright + Jest"
}
```

Infrastructure

```
# Docker Compose
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: jobbingtrack
      POSTGRES_USER: jobbingtrack
      POSTGRES_PASSWORD: jobbingtrack123

  redis:
    image: redis:7-alpine
    command: redis-server --appendonly yes

  api-gateway:
    build: ./api-gateway
    ports: ["3000:3000"]
    depends_on: [postgres, redis]
```

Sécurité

Authentification et Autorisation

- **JWT stateless** avec refresh tokens
- **Rôles granulaires** : USER, ADMIN, SUPER_ADMIN
- **Middleware d'authentification** inter-services
- **Rate limiting** configurable par endpoint

Sécurité Infrastructure

- **HTTPS obligatoire** en production
- **CORS configuré** pour les origines autorisées
- **Headers de sécurité** (HSTS, CSP, etc.)
- **Validation des entrées** à tous les niveaux

Monitoring et Observabilité

Métriques (Prometheus)

- **Temps de réponse** par service et endpoint
- **Taux d'erreur** et codes de statut HTTP

- **Utilisation CPU/Mémoire** par container
- **Nombre de requêtes** par minute/heure

Logs Structurés (Winston)

```
logger.info('User login', {  
  userId: user.id,  
  email: user.email,  
  ip: req.ip,  
  userAgent: req.get('User-Agent'),  
  timestamp: new Date().toISOString()  
});
```

Tracing Distribué (Jaeger)

- **Spans** pour chaque opération
- **Context propagation** entre services
- **Visualisation** des flux de requêtes



Déploiement

Environnement de Développement

- **Docker Compose** pour l'orchestration locale
- **Hot reload** pour le développement
- **Base de données partagée** entre services
- **Logs centralisés** pour le debugging

Production

- **Docker Swarm** ou **Kubernetes** pour la scalabilité
- **Load balancer** (Nginx) pour la distribution
- **Base de données clusterisée** (PostgreSQL)
- **Cache distribué** (Redis Cluster)

CI/CD (GitHub Actions)

```
name: CI/CD JobbingTrack  
on: [push, pull_request]  
jobs:  
  test:  
    runs-on: ubuntu-latest
```

```

services:
  postgres:
    image: postgres:15
    env:
      POSTGRES_PASSWORD: postgres
steps:
  - uses: actions/checkout@v3
  - uses: actions/setup-node@v3
  - run: npm ci
  - run: make test-integration
  - run: make test-e2e
  - run: make build

```

Gestion des Données

Synchronisation

- **Mode hors ligne** avec file d'attente
- **Synchronisation automatique** au retour en ligne
- **Gestion des conflits** avec stratégie de fusion
- **Cache intelligent** avec invalidation sélective

Backup et Restauration

- **Sauvegardes automatiques** quotidiennes
- **Stratégie 3-2-1** (3 copies, 2 médias, 1 offsite)
- **Restauration point-in-time** possible
- **Vérification d'intégrité** des sauvegardes

Évolutivité

Scalabilité Horizontale

- **Stateless services** pour la duplication facile
- **Load balancing** automatique
- **Auto-scaling** basé sur les métriques
- **Database sharding** si nécessaire

Performance

- **Cache multi-niveaux** (mémoire, Redis, base de données)
- **Optimisation des requêtes** avec indexes appropriés
- **Compression** et **CDN** pour les assets statiques
- **Lazy loading** et **code splitting**

Tests

Tests Unitaires

- **Jest** pour les tests backend
- **React Testing Library** pour les composants
- **Coverage > 80%** objectif

Tests d'Intégration

- **Tests inter-services** avec mocking
- **Tests de base de données** avec transactions
- **Tests d'authentification** end-to-end

Tests E2E (Playwright)

- **Navigation complète** de l'application
- **Tests cross-browser** (Chrome, Firefox, Safari)
- **Tests responsive** pour mobile et desktop
- **Captures d'écran** pour la régression visuelle

Configuration

Variables d'Environnement

```
# Base de données
DATABASE_URL=postgresql://jobbingtrack:password@localhost:5432/jobbingtrack

# JWT
JWT_SECRET=your-super-secret-jwt-key
JWT_REFRESH_SECRET=your-refresh-token-secret

# Services URLs
AUTH_SERVICE_URL=http://auth-service:3001
APPLICATION_SERVICE_URL=http://application-service:3002
# ... autres services

# Email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password
```


Configuration Docker

```
# docker-compose.yml
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: jobbingtrack
      POSTGRES_USER: jobbingtrack
      POSTGRES_PASSWORD: jobbingtrack123
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U jobbingtrack -d jobbingtrack"]
      interval: 10s
      timeout: 5s
      retries: 5
```

Patterns et Bonnes Pratiques

API Design

- **RESTful** avec conventions standard
- **Versioning** des endpoints (/api/v1/)
- **Pagination** pour les listes volumineuses
- **Filtering et sorting** cohérents

Gestion d'Erreurs

- **Codes d'erreur** standardisés
- **Messages d'erreur** localisés
- **Logging structuré** pour le debugging
- **Graceful degradation** en cas d'erreur

Code Quality

- **ESLint + Prettier** pour la cohérence
- **TypeScript strict** partout
- **Tests obligatoires** pour les nouvelles features
- **Documentation** des APIs avec Swagger/OpenAPI

Ressources Supplémentaires

- [Prisma Documentation](#)
 - [Express.js Guide](#)
 - [Docker Compose Reference](#)
 - [Next.js Documentation](#)
-

Cette architecture assure **scalabilité**, **maintenabilité** et **fiabilité** pour la plateforme JobbingTrack.

Navigation

-  [Index](#)
-  [Accueil](#)